
JavaScript

Interview Questions & Answers | 29 questions | Core language & runtime concepts

Contents

Fundamentals (Q1-Q5)

Functions & Scope (Q6-Q11)

Asynchronous JavaScript (Q12-Q17)

Objects, Arrays & Prototypes (Q18-Q23)

Advanced Concepts (Q24-Q29)

Fundamentals

Q1. What are the primitive data types in JavaScript?

JavaScript has seven primitive types: string, number, bigint, boolean, undefined, null, and symbol. Everything else (objects, arrays, functions) is a reference type.

- Primitives are immutable and compared by value
- Reference types are compared by reference (identity)
- `typeof null` returns "object" - a long-standing JS quirk

Q2. What is the difference between `var`, `let`, and `const`?

`var` is function-scoped and hoisted with an initial value of undefined. `let` and `const` are block-scoped and live in the "temporal dead zone" until their declaration line runs, so accessing them earlier throws a `ReferenceError`.

- `var`: function scope, can be redeclared, hoisted and initialized to undefined
- `let`: block scope, can be reassigned, not hoisted usably
- `const`: block scope, cannot be reassigned (but object contents can still mutate)

Q3. Explain hoisting in JavaScript.

Hoisting is JavaScript's behavior of moving declarations to the top of their scope during the compile phase. Function declarations are hoisted fully (usable before their definition), `var` declarations are hoisted but initialized as undefined, and `let/const` are hoisted but stay uninitialized until executed.

```
console.log(a); // undefined
var a = 5;

foo(); // works
function foo() { console.log('hi'); }

console.log(b); // ReferenceError
let b = 10;
```

Q4. What is the difference between `==` and `===`?

`===` is strict equality: it compares value and type without converting either operand. `==` is loose equality: it performs type coercion before comparing, which can produce surprising results (e.g. `"" == 0` is true). Best practice is to always use `===` unless you deliberately want coercion.

Q5. What is NaN and how do you check for it correctly?

NaN ("Not a Number") is a special numeric value representing an invalid math operation. It is the only value in JavaScript that is not equal to itself, so use `Number.isNaN(x)` or `Object.is(x, NaN)` rather than `x === NaN`, which always returns false.

```
NaN === NaN;           // false
isNaN('abc');           // true (coerces first)
Number.isNaN('abc');    // false (no coercion)
```

Functions & Scope

Q6. What is a closure and give a practical example?

A closure is a function that retains access to variables from its enclosing lexical scope even after that outer function has returned. Closures are the mechanism behind data privacy, memoization, and factory functions.

```
function makeCounter() {
  let count = 0;
  return function () {
    return ++count;
  };
}
const counter = makeCounter();
counter(); // 1
counter(); // 2
```

Q7. What is the difference between an arrow function and a regular function?

Arrow functions don't have their own 'this', 'arguments', 'super', or 'new.target' - they inherit 'this' lexically from the enclosing scope. They also cannot be used as constructors and have no prototype property. Regular functions bind their own 'this' based on how they are called.

- Arrow functions: no own this/arguments, cannot be used with 'new'
- Regular functions: dynamic this, full arguments object, hoisted if declared with function keyword

Q8. Explain the concept of 'this' in JavaScript.

'this' refers to the object that is executing the current function. Its value is determined by how a function is called, not where it is defined (call-site binding), except for arrow functions which capture 'this' lexically.

- Global context: this is the global object (or undefined in strict mode)
- Method call: this is the object the method was called on
- Constructor call (new): this is the newly created object
- call/apply/bind: this is explicitly set
- Arrow function: this is inherited from the enclosing scope

Q9. What is the difference between call, apply, and bind?

All three let you explicitly set the value of 'this' for a function. call() invokes the function immediately with arguments listed individually. apply() invokes it immediately with arguments as an array. bind() does not invoke the function - it returns a new function with 'this' permanently bound.

```
function greet(greeting) { console.log(greeting, this.name); }
const user = { name: 'Ana' };
greet.call(user, 'Hi');    // Hi Ana
greet.apply(user, ['Hi']); // Hi Ana
const bound = greet.bind(user);
bound('Hi');              // Hi Ana
```

Q10. What are higher-order functions?

A higher-order function either accepts a function as an argument, returns a function, or both. Array methods like map, filter, and reduce are common examples, as are function factories and decorators.

Q11. What is currying in JavaScript?

Currying transforms a function that takes multiple arguments into a sequence of functions that each take a single argument, enabling partial application.

```
const add = a => b => c => a + b + c;
add(1)(2)(3); // 6
const addFive = add(5);
addFive(2)(3); // 10
```

Asynchronous JavaScript

Q12. What is the event loop and how does it work?

JavaScript is single-threaded and uses an event loop to handle asynchronous operations. The call stack executes synchronous code; when an async operation (timer, I/O, promise) completes, its callback is queued. The event loop constantly checks if the call stack is empty, and if so, pushes the next queued callback onto it. Microtasks (promises) are drained before macrotasks (setTimeout, I/O).

Q13. What is the difference between a microtask and a macrotask?

Microtasks (Promise callbacks, queueMicrotask, MutationObserver) run immediately after the current synchronous code finishes and before the next macrotask. Macrotasks (setTimeout, setInterval, I/O events) are processed one at a time per event loop tick, with all pending microtasks flushed in between.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// Output: 1, 4, 3, 2
```

Q14. How do Promises work? Explain their states.

A Promise represents the eventual result of an asynchronous operation. It has three states: pending (initial), fulfilled (resolved successfully), and rejected (failed). Once settled (fulfilled or rejected), a promise's state and value are immutable, and handlers attached with .then()/catch() run as microtasks.

Q15. What is async/await and how does it relate to Promises?

async/await is syntactic sugar over Promises that lets asynchronous code read like synchronous code. An async function always returns a Promise; 'await' pauses execution of that function (without blocking the thread) until the awaited Promise settles, and errors can be caught with a plain try/catch block.

```
async function getUser(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error('Request failed');
    return await res.json();
  } catch (err) {
    console.error(err);
  }
}
```

Q16. What is the difference between Promise.all, Promise.allSettled, Promise.race, and Promise.any?

These are combinators for running multiple promises concurrently.

- Promise.all: resolves when all resolve, rejects fast on first rejection
- Promise.allSettled: waits for all to settle, returns status for each (never rejects)
- Promise.race: settles as soon as the first promise settles (fulfilled or rejected)
- Promise.any: resolves as soon as the first one fulfills, rejects only if all reject

Q17. How would you handle multiple sequential async calls that depend on each other?

Chain them with `await` inside a single async function, so each call has access to the resolved value of the previous one; wrap the chain in a `try/catch` to handle failures at any step.

```
async function checkout(cartId) {
  const cart = await getCart(cartId);
  const total = await calculateTotal(cart);
  const receipt = await charge(total);
  return receipt;
}
```

Objects, Arrays & Prototypes

Q18. What is prototypal inheritance?

Every JavaScript object has an internal link (dunder proto, exposed as `Object.getPrototypeOf`) to another object, its prototype. When a property isn't found on an object, the engine walks up the prototype chain until it finds it or reaches null. Classes in JS are syntactic sugar over this same mechanism.

Q19. What is the difference between `Object.freeze()` and `Object.seal()`?

`Object.freeze()` prevents adding, removing, or modifying properties entirely (fully immutable, shallow). `Object.seal()` prevents adding or removing properties but existing ones can still be reassigned.

Q20. Explain the difference between shallow copy and deep copy, and how to do each.

A shallow copy duplicates only the top level of an object; nested objects are still shared by reference. A deep copy recursively duplicates every nested level so no references are shared.

```
// Shallow
const shallow = { ...original };
const shallow2 = Object.assign({}, original);

// Deep
const deep = structuredClone(original);
const deep2 = JSON.parse(JSON.stringify(original)); // loses functions/dates
```

Q21. What are the differences between `map`/`filter`/`reduce`?

All three iterate an array without mutating the original.

- `map`: transforms each element, returns a new array of the same length
- `filter`: keeps elements that pass a predicate, returns a possibly shorter array
- `reduce`: accumulates all elements into a single value (number, object, array...)

```
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2); // [2,4,6,8]
const evens = nums.filter(n => n % 2 === 0); // [2,4]
const sum = nums.reduce((acc, n) => acc + n, 0); // 10
```

Q22. What is destructuring and the spread/rest operators?

Destructuring unpacks values from arrays or properties from objects into distinct variables. The spread operator (`...`) expands an iterable into individual elements; the rest operator collects remaining elements into a new array or object - same syntax, opposite direction.

```
const { name, ...rest } = { name: 'Ana', age: 30, city: 'Dubai' };
const [first, ...others] = [1, 2, 3];
const merged = { ...defaults, ...overrides };
```

Q23. What is the difference between null and undefined?

undefined means a variable has been declared but not assigned a value, or a function returned nothing. null is an explicit assignment representing 'no value' or 'empty', set intentionally by a developer. typeof undefined is 'undefined'; typeof null is 'object'.

Advanced Concepts

Q24. Explain debouncing and throttling, and the difference between them.

Both limit how often a function runs in response to frequent events (scroll, resize, keystrokes). Debouncing delays execution until a pause in events (fires once after the burst stops); throttling guarantees execution at most once per fixed interval regardless of how many events fire.

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

function throttle(fn, limit) {
  let inThrottle;
  return (...args) => {
    if (!inThrottle) {
      fn(...args);
      inThrottle = true;
      setTimeout(() => (inThrottle = false), limit);
    }
  };
}
```

Q25. What are JavaScript modules (ESM) and how do they differ from CommonJS?

ES Modules (import/export) are statically analyzable, asynchronous, and support tree-shaking; they run in strict mode by default. CommonJS (require/module.exports), used by default in Node.js, is synchronous and resolves at runtime, which makes static analysis and tree-shaking harder.

Q26. What is memoization and how would you implement it?

Memoization is an optimization technique that caches the results of expensive function calls and returns the cached result when the same inputs occur again.

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}
```

Q27. What are WeakMap and WeakSet, and why use them?

WeakMap and WeakSet hold their keys/values weakly, meaning they don't prevent garbage collection of objects used as keys. They're useful for attaching private metadata to objects (like DOM nodes) without causing memory leaks. Unlike Map/Set, they are not iterable and have no size property.

Q28. What is the difference between synchronous and asynchronous, blocking and non-blocking?

Synchronous/asynchronous describes whether code executes immediately in sequence or is deferred.

Blocking/non-blocking describes whether an operation halts the thread until it completes. Node.js uses a single thread with non-blocking I/O so it can serve many operations concurrently without spawning a thread per request.

Q29. Explain event delegation.

Event delegation attaches a single event listener to a common ancestor instead of individual listeners on every child, relying on event bubbling. This improves performance for large lists and automatically covers dynamically added child elements.

```
document.getElementById('list').addEventListener('click', (e) => {  
  if (e.target.matches('li')) {  
    console.log('Clicked item:', e.target.textContent);  
  }  
});
```